

Plum AI Assignment — OPD Claim Adjudication Tool

This repository demonstrates a complete OPD claim adjudication proof-of-concept with:

- FastAPI backend for claim validation and decisioning
- React + TypeScript + Tailwind frontend for claim submission and result display
- Document extraction pipeline that supports Word files, plain text, images, and optional PDF OCR
- Policy-driven approvals/rejections using instruction-file-based rules

What this project implements

- A backend claim adjudication engine with policy validation and rejection/approval logic
- A document extraction pipeline that can extract text from:
 - Word `.docx` files
 - plain text files (`.txt`, `.md`, `.csv`, `.html`, etc.)
 - image files (`.png`, `.jpg`, `.jpeg`, `.bmp`, `.tiff`)
 - optional PDFs when `pdf2image` is installed
- A structured adjudication response including:
 - decision status (`APPROVED`, `REJECTED`, `PARTIAL`, `MANUAL_REVIEW`)
 - approved amount
 - rejection reasons
 - confidence score
 - deductions, rejected items, and next steps
 - uploaded document summaries and policy reference metadata
- Frontend UI with file upload, document processing, and claim submission flows

System flow

1. User fills in claim details and optionally uploads supporting documents.
2. Frontend sends files to `/api/documents/process` for OCR/text extraction and document classification.
3. The backend extracts text from each file, summarises it, and merges parsable fields into the claim payload.
4. The frontend submits the merged claim to `/api/claims/submit-with-docs`.
5. Backend adjudicator evaluates the claim against policy rules and returns a decision payload.
6. Frontend displays the decision, confidence score, deductions, rejected items, document summaries, and policy references.

File structure

Root

- `backend/` — FastAPI application, request models, adjudication logic, document processing, and policy loading
- `web/` — React + Vite frontend for claim submission and result display
- `Instruction_files/` — source policy JSON and test cases from the assignment brief

Backend files

- `backend/app/main.py`
 - FastAPI app entrypoint
 - CORS settings and endpoints
 - claim submit routes and document extraction routes
- `backend/app/models.py`
 - Pydantic models for `ClaimInput`, `AdjudicationResult`, `Prescription`, `Bill`, and document payloads
 - makes optional fields safe for multipart submission
- `backend/app/policy.py`
 - loads policy JSON from `Instruction_files/`
 - checks active policy dates against treatment date
- `backend/app/document_processor.py`
 - file-type-aware extraction pipeline
 - `.docx` text extraction, plain text reading, image OCR via `easyocr`, and optional PDF OCR
 - document type detection, extraction of prescription/bill fields, and summary generation
- `backend/app/adjudicator.py`
 - core decision engine
 - validates policy, doctor registration, waiting periods, exclusions, and claim limits
 - computes approved amount, deductions, and rejects/partial approvals
 - generates structured response including `uploaded_documents` and `policy_reference`

Frontend files

- `web/src/App.tsx`
 - main React UI for entering claim and document details
 - file upload and document processing flow
 - displays adjudication result and uploaded document summaries
- `web/src/api.ts`
 - API client for `/api/claims/submit`, `/api/claims/submit-with-docs`, and `/api/documents/process`
- `web/src/types.ts`
 - TypeScript types for claim request, adjudication result, and processed document output
- `web/package.json`
 - frontend dependencies and Vite scripts

Implementation details

Document extraction pipeline

- The backend extracts document text before adjudication instead of relying solely on a fixed claim payload.
- Supported file inputs:
 - `.docx` via `python-docx`
 - plain text files via direct text read
 - images via `easyocr`
 - PDFs via `pdf2image` (if installed)
- Extracted text is used to detect document type and parse fields such as doctor details, diagnosis, medicines, and bill amounts.

- Parsed fields are merged into the claim payload, enabling richer adjudication from uploaded documents.

Adjudication rules

The backend checks claim submissions against policy rules such as:

- minimum claim amount threshold
- active policy effective date
- doctor registration number format
- waiting period enforcement for joined members and specific ailments
- excluded or cosmetic procedures
- per-claim coverage limit
- annual coverage limit
- potential manual review on multiple claims filed on the same day
- partial approval when excluded bill items are present

Confidence score calculation

The confidence score is derived from a simple heuristic:

- base score starts at 95
- subtract 12 points for each rejection or issue reason found
- add 3 points if a prescription document is present
- add 2 points if a bill document is present
- final score is clamped between 0 and 100
- returned value is normalized as a fraction between 0.0 and 1.0

This score is intended to reflect how complete the claim and supporting documents appear under current rule checks.

Running the project

Backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Frontend

```
cd web
npm install
npm run dev
```

Open the frontend at <http://localhost:5173> and keep the backend running at <http://localhost:8000>.

API endpoints

[/api/health](#)

- GET
- Returns service health status.

[/api/policy](#)

- GET
- Returns loaded policy JSON from [Instruction_files/](#).

[/api/test-cases](#)

- GET
- Returns the test cases JSON.

[/api/claims/submit](#)

- POST
- Accepts full claim JSON and returns adjudication result.

[/api/claims/submit-with-docs](#)

- POST
- Accepts multipart form-data with a claim JSON string and uploaded document files.
- Uses document processing + adjudication in one flow.

[/api/documents/extract](#)

- POST
- Accepts one file and returns raw extracted text and detected document type.

[/api/documents/process](#)

- POST
- Accepts one or more files and returns per-document parsed summaries and extracted fields.

Example request/response

Claim submission example

Request body for [/api/claims/submit](#):

```
{
  "member_id": "EMP001",
  "member_name": "Rajesh Kumar",
  "treatment_date": "2026-06-05",
  "claim_amount": 1500,
```

```
"documents": {
  "prescription": {
    "doctor_name": "Dr. Sharma",
    "doctor_reg": "KA/45678/2015",
    "diagnosis": "Viral fever",
    "medicines_prescribed": ["Paracetamol 650mg", "Vitamin C"]
  },
  "bill": {
    "consultation_fee": 1000,
    "diagnostic_tests": 500
  }
},
"member_join_date": "2025-12-01",
"previous_claims_same_day": 0
}
```

Response shape:

```
{
  "claim_id": "CLM_EMP001_1686000000",
  "decision": "APPROVED",
  "approved_amount": 1350.0,
  "rejection_reasons": [],
  "notes": "Claim approved",
  "confidence_score": 0.95,
  "next_steps": "Receive reimbursement within 7 working days",
  "flags": [],
  "deductions": {"copay": 150.0},
  "rejected_items": [],
  "uploaded_documents": [],
  "policy_reference": {
    "policy_id": "unknown",
    "matched_terms": ["Viral fever"],
    "reference_excerpt": "Checked policy coverage, waiting period, exclusions, and claim limits against current policy terms.",
    "rule_notes": [
      "Minimum claim amount must meet policy requirement.",
      "Doctor registration number is validated against policy format.",
      "Cosmetic, excluded, and waiting period conditions are checked."
    ]
  }
}
```

Document processing example

Request for `/api/documents/process` uses multipart form-data with uploaded files.

Response shape:

```
{
  "documents": [
    {
      "filename": "prescription.docx",
      "document_type": "prescription",
      "extracted_text": "Dr. Sharma\nReg. No: KA/45678/2015\nDiagnosis: Viral fever\n...",
      "summary": "Doctor: Dr. Sharma; Reg: KA/45678/2015; Diagnosis: Viral fever",
      "fields": {
        "doctor_name": "Dr. Sharma",
        "doctor_reg": "KA/45678/2015",
        "diagnosis": "Viral fever"
      }
    }
  ]
}
```

What makes this recruiter-friendly

- Shows a full-stack implementation with backend, frontend, and file-processing pipeline
- Demonstrates how raw document uploads are converted into structured claim data
- Explains the actual policy/adjudication rules implemented
- Documents the exact file structure and file responsibilities
- Highlights confidence scoring logic rather than just returning a raw decision

Notes

- This proof-of-concept is optimized for text-extractable documents and OCR images, not for production-grade document extraction.
- It is intentionally designed to show the architecture and design decisions clearly for evaluation.